

GPU 主机压力测试与监控手册

适用场景：新购 GPU 服务器验收测试、性能基线建立、稳定性验证 测试工具：nvidia-smi（监控）+ gpu-burn / cuda-samples（压力测试） 文档版本：v1.0 | 更新日期：2026-06-26

目录

- 测试前准备
- nvidia-smi 监控详解
- 压力测试方案
- 测试执行流程
- 结果分析与判定
- 常见问题排查
- 附录：一键测试脚本

一、测试前准备

1.1 硬件信息确认

```
# 查看 GPU 卡基本信息
lspci | grep -i nvidia
# 示例输出：
# 04:00.0 3D controller: NVIDIA Corporation GA102GL [RTX A6000] (rev a1)

# 查看系统 PCIe 带宽
lspci -vv -s 04:00.0 | grep LnkSta
# 示例输出：
# LnkSta: Speed 16GT/s, Width x16  - 确认跑满 PCIe 4.0 x16
```

1.2 驱动与环境检查

```
# 检查驱动版本
nvidia-smi
# 预期输出：驱动版本、CUDA 版本、GPU 状态一览

# 检查 CUDA 工具链
nvcc --version

# 检查 GPU 拓扑（多卡必做）
nvidia-smi topo -m
```

1.3 监控工具安装

```
# 安装 gpu-burn（经典压力测试工具）
git clone https://github.com/wilicc/gpu-burn.git
cd gpu-burn
make
```

```
# 或安装 cuda-samples (官方测试套件)
# 通常位于 /usr/local/cuda/samples/ 或需单独下载

# 安装 nvidia-smi (增强版监控, 可选)
# Ubuntu/Debian
sudo apt install nvidia-smi
# CentOS/Rocky
sudo yum install nvidia-smi # 或编译安装
```

1.4 测试前基线记录

```
# 记录测试前 GPU 状态
nvidia-smi -q > /tmp/gpu_baseline_$(date +%Y%m%d_%H%M%S).log

# 记录系统状态
cat /proc/cpuinfo | grep "model name" | head -1
free -h
df -h
```

二、nvidia-smi 监控详解

2.1 基础命令速查表

命令	功能	使用场景
nvidia-smi	查看 GPU 实时状态	日常巡检
nvidia-smi -l 1	每秒刷新一次	实时监控
nvidia-smi -q	详细查询所有信息	故障排查
nvidia-smi -q -d TEMPERATURE	仅查看温度信息	散热测试
nvidia-smi -q -d POWER	仅查看功耗信息	功耗验证
nvidia-smi --query-gpu=timestamp,name,pci.bus_id,temperature.gpu,utilization.gpu,utilization.memory,memory.used,memory.total,power.draw --format=csv -l 1	自定义 CSV 输出	数据记录
nvidia-smi dmon	守护进程模式监控	长期测试
nvidia-smi pmon	进程级监控	应用分析

2.2 关键监控指标解读

```
# 推荐: 测试期间持续运行的监控命令
nvidia-smi
↪ --query-gpu=timestamp,name,index,temperature.gpu,temperature.memory,utilization.gpu,utilization.memory,memory.used,memory.total,power.draw --format=csv,noheader,nounits -l 1 | tee /tmp/gpu_stress_$(date +%Y%m%d_%H%M%S).csv
```

指标说明与阈值建议

指标	说明	正常范围	告警阈值	危险阈值
temperature.gpu	GPU 核心温度	30-75°C	80°C	85°C

Table 2 – continued

指标	说明	正常范围	告警阈值	危险阈值
temperature.memory	显存温度	30-80°C	90°C	95°C
utilization.gpu	GPU 计算利用率	0-100%	—	持续 100% 需关注
utilization.memory	显存带宽利用率	0-100%	—	—
memory.used	已用显存	视业务而定	>90% 总显存	—
power.draw	实时功耗 (W)	视型号而定	>TDP 的 95%	>TDP 的 110%
power.limit	功耗限制 (W)	默认 TDP	—	—
clocks.current.sm	SM 时钟频率 (MHz)	视 Boost 状态	大幅低于标称	—
clocks.current.memory	显存频率 (MHz)	视型号而定	大幅低于标称	—
ecc.errors.corrected	ECC 可纠正错误	0	持续增长	>100/小时
ecc.errors.uncorrected	ECC 不可纠正错误	必须为 0	>0	立即停机
fan.speed	风扇转速 (%)	30-85%	>90%	100% 持续运行
pstate	性能状态	P0(最高)-P12(最低)	负载下非 P0	—

2.3 多卡监控技巧

```
# 查看指定 GPU (如 GPU 0)
nvidia-smi -i 0

# 查看所有 GPU 的拓扑关系 (多卡通信效率关键)
nvidia-smi topo -m

# 输出示例:
#      GPU0      GPU1      GPU2      GPU3      CPU Affinity
# GPU0   X        NV4       NV4       NV4       0-31,64-95
# GPU1  NV4        X        NV4       NV4       0-31,64-95
# NV4 = NVLink 4th Gen, 数值越大带宽越高

# 查看 NVLink 状态
nvidia-smi nvlink --status
nvidia-smi nvlink --get_cumulative_replay_count

# 设置持久化模式 (推荐开机启用, 避免驱动初始化延迟)
sudo nvidia-smi -pm 1

# 设置性能模式 (测试前)
sudo nvidia-smi -ac 877,1530 # 锁定显存/核心频率 (根据卡型号调整)
```

2.4 进程级监控

```
# 查看 GPU 上的进程
nvidia-smi pmon -s um -o T

# 输出字段说明:
# gpu          - GPU 编号
# pid          - 进程 ID
# type         - 进程类型 (C= 计算, G= 图形)
# sm           - SM 利用率
# mem          - 显存带宽利用率
```

```
# enc      - 编码器利用率
# dec      - 解码器利用率
# jpg      - JPEG 利用率
# ofa      - OFA 利用率
# command  - 进程名
```

三、压力测试方案

3.1 测试策略总览

测试类型	工具	目的	时长	通过标准
显存压力测试	gpu-burn	验证显存稳定性	30min-2h	无 ECC 错误、无降频
计算压力测试	gpu-burn / cuda-samples	验证 CUDA 核心稳定性	30min-2h	温度 <85°C、无计算错误
带宽压力测试	cuda-samples (bandwidthTest)	验证 HBM/GDDR 显存带宽	15min	达到标称带宽 90%+
PCIe 带宽测试	cuda-samples (p2pBandwidth)	验证 PCIe 传输效率	15min	达到标称带宽 80%+
多卡 P2P 测试	cuda-samples (p2pBandwidth)	验证 NVLink/PCIe P2P	15min	NVLink 达到标称值
长时间稳定性	gpu-burn	验证散热与长期稳定性	24h+	无错误、温度稳定
功耗测试	gpu-burn + nvidia-smi	验证电源与散热能力	30min	达到 TDP、温度可控

3.2 gpu-burn 详解

```
# 基础用法：所有 GPU 测试，默认 10 分钟
cd gpu-burn
./gpu_burn

# 常用参数说明：
# -d <sec>    测试时长（秒），默认 600
# -m <percent> 显存使用百分比，默认 90%
# -tc <temp>   温度限制，超过则降频
# -i <gpu_id>  指定 GPU 测试
# -c <path>    使用自定义 CUDA 内核
# -l          列出可用 GPU

# 示例 1：所有 GPU，90% 显存，测试 30 分钟
./gpu_burn -d 1800 -m 90

# 示例 2：仅测试 GPU 0，使用 95% 显存，测试 2 小时
./gpu_burn -d 7200 -m 95 -i 0

# 示例 3：8 卡服务器全卡测试，24 小时稳定性验证
./gpu_burn -d 86400 -m 90
```

```
# 示例 4: 带温度限制的测试 (超过 80°C 自动降频保护)
./gpu_burn -d 3600 -tc 80
```

gpu-burn 输出解读:

```
GPU 0: OK   ← 正常
GPU 1: OK
GPU 2: ERROR: 0.000000 ← 计算错误!
GPU 3: OK
```

3.3 CUDA Samples 测试

```
# 进入 CUDA Samples 目录 (路径可能不同)
cd /usr/local/cuda/samples/1_Uutilities/

# 编译测试程序
make

# 1. 设备信息查询
cd deviceQuery
./deviceQuery
# 关键检查: CUDA Capability、总显存、多处理器数量、Warp 大小

# 2. 带宽测试
cd ../bandwidthTest
./bandwidthTest --mode=quick
./bandwidthTest --mode=range --start=1024 --end=104857600 --increment=1048576
# 关键指标: Device to Device 带宽应达到标称值 90% 以上

# 3. PCIe 带宽测试 (H2D/D2H)
./bandwidthTest --device=0 --memory=pinned
# 关键指标: H2D/D2H 应达到 PCIe 理论带宽的 80%+

# 4. P2P 带宽测试 (多卡)
cd ../p2pBandwidthLatencyTest
./p2pBandwidthLatencyTest
# 关键指标: NVLink P2P 应达到标称值 (如 NVLink 4: 90GB/s)

# 5. 计算压力测试
cd ../cudaTensorCoreGemm
make
./cudaTensorCoreGemm
# 或使用 matrixMul 等计算密集型测试
```

3.4 自定义 CUDA 压力测试 (进阶)

```
# 创建自定义压力测试脚本: stress_cuda.cu
cat > stress_cuda.cu << 'EOF'
#include <cuda_runtime.h>
#include <stdio.h>
#include <time.h>
```

```

#define CHECK_CUDA(call)    do {          cudaError_t err = call;          if (err != cudaSuccess) {
↪   fprintf(stderr, "CUDA error at %s:%d: %s\n",          __FILE__, __LINE__,
↪   cudaGetErrorString(err));          exit(1);          }          } while(0)

__global__ void stress_kernel(float *a, float *b, float *c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        for (int i = 0; i < 1000; i++) {
            c[idx] = a[idx] * b[idx] + c[idx];
            c[idx] = sqrtf(fabsf(c[idx]));
        }
    }
}

int main(int argc, char **argv) {
    int gpu_id = 0;
    int duration = 60; // 默认 60 秒

    if (argc > 1) gpu_id = atoi(argv[1]);
    if (argc > 2) duration = atoi(argv[2]);

    CHECK_CUDA(cudaSetDevice(gpu_id));

    cudaDeviceProp prop;
    CHECK_CUDA(cudaGetDeviceProperties(&prop, gpu_id));
    printf("Testing GPU %d: %s\n", gpu_id, prop.name);

    size_t free_mem, total_mem;
    CHECK_CUDA(cudaMemGetInfo(&free_mem, &total_mem));

    // 使用 90% 可用显存
    size_t mem_size = (size_t)(free_mem * 0.9);
    int n = mem_size / sizeof(float) / 3; // 3 个数组

    printf("Using %.2f GB memory (%d elements)\n",
           (float)(n * 3 * sizeof(float)) / (1024*1024*1024), n);

    float *d_a, *d_b, *d_c;
    CHECK_CUDA(cudaMalloc(&d_a, n * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_b, n * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_c, n * sizeof(float)));

    // 初始化
    CHECK_CUDA(cudaMemset(d_a, 1, n * sizeof(float)));
    CHECK_CUDA(cudaMemset(d_b, 2, n * sizeof(float)));
    CHECK_CUDA(cudaMemset(d_c, 0, n * sizeof(float)));

    int threads = 256;
    int blocks = (n + threads - 1) / threads;

```

```

printf("Running stress test for %d seconds...\n", duration);
time_t start = time(NULL);
int iterations = 0;

while (time(NULL) - start < duration) {
    stress_kernel<<<blocks, threads>>>(d_a, d_b, d_c, n);
    CHECK_CUDA(cudaDeviceSynchronize());
    iterations++;

    if (iterations % 10 == 0) {
        printf("Progress: %ld seconds, %d iterations\n",
            time(NULL) - start, iterations);
    }
}

printf("\nCompleted %d iterations in %d seconds\n",
    iterations, duration);

cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);

return 0;
}
EOF

# 编译
nvcc -o stress_cuda stress_cuda.cu -arch=sm_80 # 根据 GPU 架构调整

# 运行: 测试 GPU 0, 持续 300 秒
./stress_cuda 0 300

```

四、测试执行流程

4.1 标准验收测试流程（建议）

```

#!/bin/bash
# =====
# GPU 服务器验收测试脚本
# 执行时间: 约 3-4 小时 (不含 24h 稳定性测试)
# =====

TEST_LOG="/tmp/gpu_test_$(date +%Y%m%d_%H%M%S)"
mkdir -p $TEST_LOG

echo "===== GPU 服务器验收测试开始 ====="
echo "测试时间: $(date)"
echo "测试日志: $TEST_LOG"
echo ""

```

```

# 步骤 1: 环境检查
echo "[1/8] 环境检查..."
nvidia-smi > $TEST_LOG/01_nvidia_smi.txt 2>&1
nvidia-smi -q > $TEST_LOG/01_nvidia_smi_query.txt 2>&1
nvidia-smi topo -m > $TEST_LOG/01_topo.txt 2>&1
nvcc --version > $TEST_LOG/01_cuda_version.txt 2>&1
cat /proc/driver/nvidia/version > $TEST_LOG/01_driver_version.txt 2>&1
echo "完成 $\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
↪ 2713.png}}}}}$S"
echo ""

# 步骤 2: 设备信息查询
echo "[2/8] 设备信息查询..."
cd /usr/local/cuda/samples/1_Uutilities/deviceQuery
./deviceQuery > $TEST_LOG/02_deviceQuery.txt 2>&1
echo "完成 $\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
↪ 2713.png}}}}}$S"
echo ""

# 步骤 3: 显存带宽测试
echo "[3/8] 显存带宽测试..."
cd /usr/local/cuda/samples/1_Uutilities/bandwidthTest
./bandwidthTest --mode=range --start=1024 --end=104857600 --increment=1048576 > $TEST_LOG/
↪ 03_bandwidth.txt 2>&1
echo "完成 $\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
↪ 2713.png}}}}}$S"
echo ""

# 步骤 4: PCIe 带宽测试
echo "[4/8] PCIe 带宽测试..."
./bandwidthTest --device=all --memory=pinned > $TEST_LOG/04_pcie_bandwidth.txt 2>&1
echo "完成 $\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
↪ 2713.png}}}}}$S"
echo ""

# 步骤 5: P2P 带宽测试 (多卡)
echo "[5/8] P2P 带宽测试..."
cd /usr/local/cuda/samples/1_Uutilities/p2pBandwidthLatencyTest
./p2pBandwidthLatencyTest > $TEST_LOG/05_p2p.txt 2>&1 || echo "P2P 测试跳过 (单卡或 P2P 不支持)"
echo "完成 $\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
↪ 2713.png}}}}}$S"
echo ""

# 步骤 6: 显存压力测试 (30 分钟)
echo "[6/8] 显存压力测试 (30 分钟) ..."
echo "开始时间: $(date)"
cd /path/to/gpu-burn # 修改为你的 gpu-burn 路径
./gpu_burn -d 1800 -m 90 > $TEST_LOG/06_gpu_burn.txt 2>&1 &
GPU_BURN_PID=$!

# 同时启动监控

```



```

nvidia-smi
↪ --query-gpu=timestamp,name,temperature.gpu,temperature.memory,utilization.gpu,utilization.memory,memory.used,memory.free
↪ --format=csv -l 1 1 > $TEST_LOG/06_monitor.csv 2>&1 &
MONITOR_PID=$!

wait $GPU_BURN_PID
kill $MONITOR_PID 2>/dev/null
echo "完成 $$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/doc/miner/emoji/
↪ 2713.png}}}}}$\$"
echo ""

# 步骤 7: 计算压力测试 (30 分钟)
echo "[7/8] 计算压力测试 (30 分钟) ..."
echo "开始时间: $(date)"
# 使用自定义 CUDA 测试或 gpu-burn
cd /path/to/gpu-burn
./gpu_burn -d 1800 -m 50 > $TEST_LOG/07_compute_stress.txt 2>&1 &
GPU_BURN_PID=$!

nvidia-smi
↪ --query-gpu=timestamp,name,temperature.gpu,utilization.gpu,power.draw,clocks.current.sm,ecc.errors.corrected,ecc.errors.uncorrected
↪ --format=csv -l 1 1 > $TEST_LOG/07_monitor.csv 2>&1 &
MONITOR_PID=$!

wait $GPU_BURN_PID
kill $MONITOR_PID 2>/dev/null
echo "完成 $$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/doc/miner/emoji/
↪ 2713.png}}}}}$\$"
echo ""

# 步骤 8: 结果汇总
echo "[8/8] 结果汇总..."
echo "===== 测试结果汇总 =====" > $TEST_LOG/08_summary.txt
echo "测试时间: $(date)" >> $TEST_LOG/08_summary.txt
echo "" >> $TEST_LOG/08_summary.txt

# 检查 ECC 错误
echo "--- ECC 错误检查 ---" >> $TEST_LOG/08_summary.txt
nvidia-smi -q -d ECC | grep -A 5 "Error" >> $TEST_LOG/08_summary.txt
echo "" >> $TEST_LOG/08_summary.txt

# 检查温度峰值
echo "--- 温度统计 ---" >> $TEST_LOG/08_summary.txt
if [ -f "$TEST_LOG/06_monitor.csv" ]; then
    echo "GPU 温度峰值:"
    tail -n +2 $TEST_LOG/06_monitor.csv | awk -F',' '{print $3}' | sort -n | tail -1
    echo "显存温度峰值:"
    tail -n +2 $TEST_LOG/06_monitor.csv | awk -F',' '{print $4}' | sort -n | tail -1
fi >> $TEST_LOG/08_summary.txt
echo "" >> $TEST_LOG/08_summary.txt

```

```
# 检查 gpu-burn 结果
echo "--- gpu-burn 结果 ---" >> $TEST_LOG/08_summary.txt
grep -E "(ERROR|OK)" $TEST_LOG/06_gpu_burn.txt | tail -20 >> $TEST_LOG/08_summary.txt

echo "测试完成! 日志目录: $TEST_LOG"
echo "请查看 $TEST_LOG/08_summary.txt 获取汇总结果"
```

4.2 实时监控终端布局建议

推荐开 3 个终端窗口:

终端 1 - GPU 概览监控:

```
watch -n 1 nvidia-smi
```

终端 2 - 详细指标 CSV 记录:

```
nvidia-smi
↵ --query-gpu=timestamp,name,temperature.gpu,temperature.memory,utilization.gpu,utilization.memory,memory.used,memory.free
↵ --format=csv -l 1 | tee gpu_monitor_$(date +%Y%m%d).csv
```

终端 3 - 进程监控:

```
nvidia-smi pmon -s um -o T
```

五、结果分析与判定

5.1 验收通过标准

检查项	通过标准	备注
显存测试	gpu-burn 30 分钟无 ERROR	所有 GPU 显示 OK
计算测试	30 分钟无 ECC 不可纠正错误	<code>ecc.errors.uncorrected = 0</code>
温度控制	GPU 核心 <85°C, 显存 <95°C	持续超过需检查散热
功耗达标	达到 TDP 的 90% 以上	未达标可能电源或驱动问题
频率稳定	负载下保持 Boost 频率	大幅降频说明散热或供电不足
显存带宽	达到标称带宽 90%	如 H100 标称 3.35TB/s, 实测 >3.0TB/s
PCIe 带宽	H2D/D2H 达到理论值 80%	PCIe 4.0 x16: ~32GB/s
P2P 带宽	NVLink 达到标称值	NVLink 4: ~90GB/s
风扇转速	<90% (满载)	100% 持续说明散热不足
稳定性	24 小时测试无错误	关键业务服务器必做

5.2 结果分析示例

```
# 快速分析监控 CSV
# 1. 查看温度峰值
cat gpu_monitor.csv | awk -F',' '{print $3}' | grep -v temp | sort -n | tail -1

# 2. 查看是否有 ECC 错误
```

```
cat gpu_monitor.csv | awk -F',' '{print $12,"$13}' | grep -v ecc | sort | uniq -c

# 3. 查看功耗是否达标
cat gpu_monitor.csv | awk -F',' '{print $10}' | grep -v power | awk '{sum+=$1; count++;} END {print "Avg
↪ Power: " sum/count " W"}'

# 4. 查看频率是否稳定
cat gpu_monitor.csv | awk -F',' '{print $11}' | grep -v clocks | sort -n | uniq -c
```

5.3 测试报告模板

```
# GPU 服务器压力测试报告

## 基本信息
- 测试日期: 2026-06-26
- 测试人员: [姓名]
- 服务器型号: [品牌型号]
- GPU 型号: [如 NVIDIA H100 80GB PCIe]
- GPU 数量: [8]
- 驱动版本: [535.XX]
- CUDA 版本: [12.2]

## 测试项目与结果

| 测试项目 | 时长 | 结果 | 备注 |
|:---|:---|:---|:---|
| 显存压力测试 | 30min | [通过/失败] | 温度峰值: XX°C |
| 计算压力测试 | 30min | [通过/失败] | 功耗峰值: XXXW |
| 显存带宽测试 | 15min | [通过/失败] | 实测: X.XX TB/s |
| PCIe 带宽测试 | 15min | [通过/失败] | H2D: XX GB/s |
| P2P 带宽测试 | 15min | [通过/失败] | NVLink: XX GB/s |
| 24h 稳定性测试 | 24h | [通过/失败] | ECC 错误: 0 |

## 关键指标汇总
- GPU 最高温度: XX°C
- 显存最高温度: XX°C
- 最高功耗: XXXW (TDP: XXXW)
- 最低 SM 频率: XXXX MHz
- ECC 可纠正错误: X
- ECC 不可纠正错误: 0

## 结论
[通过 / 有条件通过 / 不通过]

## 建议
[如有问题, 列出改进建议]
```

六、常见问题排查

6.1 nvidia-smi 报错

问题	原因	解决方案
NVIDIA-SMI has failed... No devices were found	驱动未加载 GPU 未识别	sudo modprobe nvidia 或重装驱动 检查 PCIe 连接、BIOS 设置、驱动兼容性
Permission denied Failed to initialize NVML	权限不足 NVML 库问题	sudo nvidia-smi 或加入 video 组 重启 nvidia-persistenced 服务

6.2 温度异常

```
# 检查风扇状态
nvidia-smi -q -d TEMPERATURE | grep -A 10 "Fan Speed"

# 手动设置风扇转速（如支持）
sudo nvidia-smi -pm 1
sudo nvidia-smi -pl 250 # 限制功耗 250W

# 检查机箱风道
nvidia-smi -q | grep "Airflow"
```

6.3 性能不达标

现象	排查方向	检查命令
显存带宽低	PCIe 降速	lspci -vv -s <bus_id> \ grep LnkSta
计算性能低	温度降频	nvidia-smi -q \ grep "Clock Throttle"
功耗上不去	电源限制	nvidia-smi -q \ grep "Power Limit"
P2P 带宽低	NVLink 未启用	nvidia-smi nvlink --status

6.4 ECC 错误处理

```
# 查看 ECC 错误详情
nvidia-smi -q -d ECC

# 如果有不可纠正错误，必须立即处理
# 1. 记录错误数量
# 2. 停止测试
# 3. 联系厂商更换 GPU

# 重置 ECC 计数（仅可纠正错误，测试前清零）
sudo nvidia-smi --ecc-reset=0 # 重置 GPU 0
```

七、附录：一键测试脚本

7.1 完整一键测试脚本

```
#!/bin/bash
#####
# GPU 服务器一键压力测试脚本
# 作者: [你的名字]
# 日期: 2026-06-26
# 功能: 自动化执行 GPU 验收测试, 包含监控、测试、报告生成
# 用法: sudo ./gpu_stress_test.sh [测试时长 (分钟)] [显存使用百分比]
#####

set -e

# 参数设置
DURATION=${1:-30}          # 默认 30 分钟
MEM_PERCENT=${2:-90}       # 默认使用 90% 显存
GPU_BURN_PATH="/opt/gpu-burn" # 修改为你的 gpu-burn 路径
LOG_DIR="/tmp/gpu_test_$(date +%Y%m%d_%H%M%S)"

# 颜色定义
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m'

# 日志函数
log_info() { echo -e "${GREEN}[INFO]${NC} $1"; }
log_warn() { echo -e "${YELLOW}[WARN]${NC} $1"; }
log_error() { echo -e "${RED}[ERROR]${NC} $1"; }

# 检查 root 权限
check_root() {
    if [ "$EUID" -ne 0 ]; then
        log_error "请使用 sudo 运行此脚本"
        exit 1
    fi
}

# 检查依赖
check_dependencies() {
    log_info "检查依赖..."

    if ! command -v nvidia-smi &> /dev/null; then
        log_error "nvidia-smi 未找到, 请安装 NVIDIA 驱动"
        exit 1
    fi

    if [ ! -f "$GPU_BURN_PATH/gpu_burn" ]; then
        log_error "gpu-burn 未找到, 请检查路径: $GPU_BURN_PATH"
        exit 1
    fi
}
```

```

fi

log_info "依赖检查通过 ${\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
↪ emoji/2713.png}}}}}}}"$"$

}

# 获取 GPU 信息
get_gpu_info() {
    log_info " 获取 GPU 信息..."

    GPU_COUNT=$(nvidia-smi -L | wc -l)
    GPU_NAME=$(nvidia-smi -L | head -1 | awk -F': ' '{print $2}' | awk -F'(' '{print $1}')')
    DRIVER_VERSION=$(nvidia-smi | grep "Driver Version" | awk '{print $3}')
    CUDA_VERSION=$(nvidia-smi | grep "CUDA Version" | awk '{print $9}')

    log_info "检测到 $GPU_COUNT 块 GPU: $GPU_NAME"
    log_info "驱动版本: $DRIVER_VERSION, CUDA 版本: $CUDA_VERSION"

    # 记录到日志
    cat > $LOG_DIR/gpu_info.txt << EOF
GPU 数量: $GPU_COUNT
GPU 型号: $GPU_NAME
驱动版本: $DRIVER_VERSION
CUDA 版本: $CUDA_VERSION
测试时间: $(date)
测试时长: ${DURATION} 分钟
显存使用: ${MEM_PERCENT}%
EOF
}

# 启动监控
start_monitor() {
    log_info " 启动后台监控..."

    nvidia-smi
    ↪ --query-gpu=timestamp,name,index,temperature.gpu,temperature.memory,utilization.gpu,utilization.memory,memory.used
    ↪ --format=csv,noheader,nounits -l 1 > $LOG_DIR/monitor.csv 2>&1 &

    MONITOR_PID=$!
    echo $MONITOR_PID > $LOG_DIR/monitor.pid
    log_info "监控进程 PID: $MONITOR_PID"
}

# 停止监控
stop_monitor() {
    if [ -f "$LOG_DIR/monitor.pid" ]; then
        MONITOR_PID=$(cat $LOG_DIR/monitor.pid)
        kill $MONITOR_PID 2>/dev/null || true
        rm -f $LOG_DIR/monitor.pid
        log_info " 监控已停止"
    fi
}

```

```

}

# 执行压力测试
run_stress_test() {
    log_info "开始压力测试: ${DURATION}分钟, 显存使用${MEM_PERCENT}%"
    log_info "开始时间: $(date)"

    SECONDS=$((DURATION * 60))

    cd $GPU_BURN_PATH
    ./gpu_burn -d $SECONDS -m $MEM_PERCENT > $LOG_DIR/gpu_burn.log 2>&1 &
    GPU_BURN_PID=$!

    log_info "gpu-burn PID: $GPU_BURN_PID"

    # 等待测试完成, 同时显示进度
    echo ""
    for ((i=0; i<DURATION; i++)); do
        sleep 60
        PROGRESS=$(( (i + 1) * 100 / DURATION ))
        printf "\r 进度: [%-50s] %d%% (%d/%d 分钟)" $(printf '%.0s' $(seq 1 $((PROGRESS /
        ↪ 2)))) $PROGRESS $((i+1)) $DURATION
    done
    echo ""

    wait $GPU_BURN_PID
    log_info "测试完成: $(date)"
}

# 分析结果
analyze_results() {
    log_info " 分析测试结果..."

    # 检查 gpu-burn 结果
    if grep -q "ERROR" $LOG_DIR/gpu_burn.log; then
        log_error "gpu-burn 检测到计算错误! "
        grep "ERROR" $LOG_DIR/gpu_burn.log > $LOG_DIR/errors.txt
    else
        log_info "gpu-burn 测试通过, 无计算错误  ${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/
        ↪ usr/share/docminer/emoji/2713.png}}}}}$"
    fi

    # 温度分析
    if [ -f "$LOG_DIR/monitor.csv" ]; then
        MAX_GPU_TEMP=$(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{print $4}' | sort -n | tail -1)
        MAX_MEM_TEMP=$(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{print $5}' | sort -n | tail -1)
        MAX_POWER=$(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{print $11}' | sort -n | tail -1)
        AVG_UTIL=$(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{sum+=$6; count++} END {printf "%.1f",
        ↪ sum/count}')

        log_info "GPU 最高温度: ${MAX_GPU_TEMP}°C"
    fi
}

```

```

log_info "显存最高温度: ${MAX_MEM_TEMP}°C"
log_info "最高功耗: ${MAX_POWER}W"
log_info "平均 GPU 利用率: ${AVG_UTIL}%"

# 温度判定
if [ "${MAX_GPU_TEMP%.*}" -gt 85 ]; then
    log_warn "GPU 温度超过 85°C, 请检查散热!"
elif [ "${MAX_GPU_TEMP%.*}" -gt 80 ]; then
    log_warn "GPU 温度超过 80°C, 建议关注散热"
else
    log_info "温度正常 ${vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/
    ↪ docminer/emoji/2713.png}}}}}$"
fi

# ECC 错误检查
ECC_UNCORR=$(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{sum+=$16} END {print sum}')
if [ "$ECC_UNCORR" -gt 0 ]; then
    log_error "检测到 $ECC_UNCORR 个不可纠正 ECC 错误! GPU 可能存在硬件问题!"
else
    log_info "无不可纠正 ECC 错误 ${vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/
    ↪ share/docminer/emoji/2713.png}}}}}$"
fi
fi
}

# 生成报告
generate_report() {
    log_info "生成测试报告..."

    cat > $LOG_DIR/TEST_REPORT.txt << EOF
=====
GPU 服务器压力测试报告
=====

测试时间: $(date)
日志目录: $LOG_DIR

--- 硬件信息 ---
$(cat $LOG_DIR/gpu_info.txt)

--- 测试结果 ---
$(if grep -q "ERROR" $LOG_DIR/gpu_burn.log 2>/dev/null; then echo " 状态: FAILED - 存在计算错误"; else
    ↪ echo " 状态: PASSED - 无计算错误"; fi)

--- 关键指标 ---
$(if [ -f "$LOG_DIR/monitor.csv" ]; then
    echo "GPU 最高温度: $(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{print $4}' | sort -n | tail -1)°C"
    echo " 显存最高温度: $(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{print $5}' | sort -n | tail -1)°C"
    echo " 最高功耗: $(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{print $11}' | sort -n | tail -1)W"
    echo " 平均 GPU 利用率: $(tail -n +2 $LOG_DIR/monitor.csv | awk -F',' '{sum+=$6; count++;} END {printf
    ↪ "%.1f%%", sum/count}')"
fi)

```



```

--- 文件清单 ---
$(ls -la $LOG_DIR/)

=====
EOF

    log_info "报告已生成: $LOG_DIR/TEST_REPORT.txt"
}

# 主函数
main() {
    echo "=====
    echo "      GPU 服务器压力测试脚本"
    echo "=====
    echo ""

    mkdir -p $LOG_DIR

    check_root
    check_dependencies
    get_gpu_info
    start_monitor

    # 捕获中断信号, 确保清理
    trap 'log_warn " 收到中断信号, 正在清理..."; stop_monitor; exit 1' INT TERM

    run_stress_test
    stop_monitor
    analyze_results
    generate_report

    echo ""
    echo "=====
    log_info " 测试全部完成! "
    log_info "日志目录: $LOG_DIR"
    log_info "报告文件: $LOG_DIR/TEST_REPORT.txt"
    echo "=====
}

# 运行
main "$@"

```

7.2 使用说明

```

# 1. 保存脚本
sudo tee /usr/local/bin/gpu_stress_test.sh > /dev/null << 'SCRIPT'
# [粘贴上面脚本内容]
SCRIPT

# 2. 添加执行权限

```

```
sudo chmod +x /usr/local/bin/gpu_stress_test.sh

# 3. 运行测试 (默认 30 分钟, 90% 显存)
sudo gpu_stress_test.sh

# 4. 自定义参数: 测试 60 分钟, 使用 95% 显存
sudo gpu_stress_test.sh 60 95

# 5. 查看实时日志
tail -f /tmp/gpu_test_*/gpu_burn.log

# 6. 查看监控数据
cat /tmp/gpu_test_*/monitor.csv | column -t -s,
```

参考文档

- [NVIDIA System Management Interface \(nvidia-smi\) 官方文档](#)
- [gpu-burn GitHub 仓库](#)
- [CUDA Samples 文档](#)
- [NVIDIA GPU 温度与功耗规范](#)

最后更新: 2026-06-26 维护者: [你的名字] 如有问题: 请检查日志目录中的详细输出, 或联系硬件供应商